

Lab Experiment 5 : Linear Regression through Gradient Descent

Aim

To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on a real world dataset.

Objectives

- To understand the concept of Gradient Descent for optimizing a Linear Regression model.
- To preprocess the dataset before model training.
- To implement Linear Regression using Gradient Descent.
- To analyze the convergence of the loss function during training.
- To evaluate the model using standard regression metrics.

Task 1: Download and Load the Dataset

We will download the Student Performance dataset directly from the UCI repository using pandas.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings("ignore")

# Set random seed for reproducibility
np.random.seed(42)

# Load the dataset (student-mat.csv)
df = pd.read_csv("student-mat.csv", sep=";")

print("Dataset loaded successfully!")
print(f"Shape: {df.shape}")
print(f"\nFirst 5 rows:\n{df.head()}")
print(f"\nColumn names:\n{df.columns.tolist()}")
```

Dataset loaded successfully!

Shape: (395, 33)

First 5 rows:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob
0	GP	F	18	U	GT3	A	4	4	at_home	teacher
1	GP	F	17	U	GT3	T	1	1	at_home	other
2	GP	F	15	U	LE3	T	1	1	at_home	other
3	GP	F	15	U	GT3	T	4	2	health	services
4	GP	F	16	U	GT3	T	3	3	other	other

	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	4	3	4	1	1	3	6	5	6	6
1	5	3	3	1	1	3	4	5	5	6
2	4	3	2	2	3	3	10	7	8	10
3	3	2	2	1	1	5	2	15	14	15
4	4	3	2	1	2	5	4	6	10	10

[5 rows x 33 columns]

Column names:

['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu', 'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime', 'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

Task 2: Data Preprocessing

In this section, we will:

- Check for missing values
- Explore the dataset structure
- Encode categorical variables
- Scale numerical features

```
In [2]: # Check for missing values
print("Missing values per column:")
print(df.isnull().sum())
print(f"\nTotal missing values: {df.isnull().sum().sum()}")

# Dataset info
print("\nDataset Info:")
print(df.dtypes)

# Summary statistics
```

```
print("\nSummary Statistics:")  
print(df.describe())
```

```
Missing values per column:
school      0
sex         0
age         0
address     0
famsize     0
Pstatus     0
Medu        0
Fedu        0
Mjob        0
Fjob        0
reason      0
guardian    0
traveltime  0
studytime   0
failures    0
schoolsup   0
famsup      0
paid        0
activities  0
nursery     0
higher      0
internet    0
romantic    0
famrel      0
freetime    0
goout       0
Dalc        0
Walc        0
health      0
absences    0
G1          0
G2          0
G3          0
dtype: int64
```

Total missing values: 0

```
Dataset Info:
school      str
sex         str
age         int64
address     str
famsize     str
Pstatus     str
Medu        int64
Fedu        int64
Mjob        str
Fjob        str
reason      str
guardian    str
traveltime  int64
studytime   int64
failures    int64
schoolsup   str
famsup      str
```

```

paid          str
activities    str
nursery       str
higher        str
internet      str
romantic      str
famrel        int64
freetime      int64
goout         int64
Dalc          int64
Walc          int64
health        int64
absences      int64
G1            int64
G2            int64
G3            int64
dtype: object

```

Summary Statistics:

	age	Medu	Fedu	traveltime	studytime	failure
s \ count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.33417
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.74365
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.00000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.00000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.00000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.00000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.00000

	famrel	freetime	goout	Dalc	Walc	healt
h \ count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	3.944304	3.235443	3.108861	1.481013	2.291139	3.55443
std	0.896659	0.998862	1.113278	0.890741	1.287897	1.39030
min	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
25%	4.000000	3.000000	2.000000	1.000000	1.000000	3.00000
50%	4.000000	3.000000	3.000000	1.000000	2.000000	4.00000
75%	5.000000	4.000000	4.000000	2.000000	3.000000	5.00000
max	5.000000	5.000000	5.000000	5.000000	5.000000	5.00000

	absences	G1	G2	G3
count	395.000000	395.000000	395.000000	395.000000
mean	5.708861	10.908861	10.713924	10.415190
std	8.003096	3.319195	3.761505	4.581443
min	0.000000	3.000000	0.000000	0.000000
25%	0.000000	8.000000	9.000000	8.000000
50%	4.000000	11.000000	11.000000	11.000000
75%	8.000000	13.000000	13.000000	14.000000
max	75.000000	19.000000	19.000000	20.000000

Handling Categorical Variables

We will use OneHotEncoder for categorical features and StandardScaler for numerical features.

```
In [3]: # Identify categorical and numerical columns
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()
numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()

print(f"Categorical columns ({len(categorical_cols)}): {categorical_cols}")
print(f"\nNumerical columns ({len(numerical_cols)}): {numerical_cols}")
```

Categorical columns (17): ['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob', 'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']

Numerical columns (16): ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

```
In [4]: # Define target variable (G3 - final grade)
target = "G3"

# Remove G1 and G2 to avoid data leakage (G3 is our target)
features = [col for col in numerical_cols if col not in ["G1", "G2", "G3"]]

print(f"Features to use ({len(features)}): {features}")
print(f"Target: {target}")
```

Features to use (30): ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob', 'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']
Target: G3

Task 3: Split Dataset into Training and Testing Sets

We will use 80% for training and 20% for testing.

```
In [5]: X = df[features]
y = df[target]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

Training set shape: (316, 30)

Testing set shape: (79, 30)

Task 4: Implement Linear Regression using Gradient Descent

We will implement Linear Regression from scratch using the Gradient Descent optimization algorithm.

```
In [6]: class LinearRegressionGD:
        """
        Linear Regression using Gradient Descent
        """
        def __init__(self, learning_rate=0.01, n_iterations=1000):
            self.learning_rate = learning_rate
            self.n_iterations = n_iterations
            self.weights = None
            self.bias = None
            self.loss_history = []

        def _compute_loss(self, X, y):
            n = len(y)
            predictions = X.dot(self.weights) + self.bias
            loss = (1 / (2 * n)) * np.sum((predictions - y) ** 2)
            return loss

        def fit(self, X, y):
            n_samples, n_features = X.shape
            self.weights = np.zeros(n_features)
            self.bias = 0
            self.loss_history = []

            for i in range(self.n_iterations):
                predictions = X.dot(self.weights) + self.bias

                # Compute gradients
                dw = (1 / n_samples) * X.T.dot(predictions - y)
                db = (1 / n_samples) * np.sum(predictions - y)

                # Update parameters
                self.weights -= self.learning_rate * dw
                self.bias -= self.learning_rate * db

                # Record loss
                loss = self._compute_loss(X, y)
                self.loss_history.append(loss)

            return self
```

```
def predict(self, X):
    return X.dot(self.weights) + self.bias

print("LinearRegressionGD class defined successfully!")
```

LinearRegressionGD class defined successfully!

```
In [7]: # Create preprocessing pipeline
numeric_features = [col for col in features if col in numerical_cols]
categorical_features = [col for col in features if col in categorical_cols]

numeric_transformer = Pipeline(steps=[
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("onehot", OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

# Fit preprocessor on training data and transform both
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print(f"Processed training shape: {X_train_processed.shape}")
print(f"Processed testing shape: {X_test_processed.shape}")
```

Processed training shape: (316, 39)

Processed testing shape: (79, 39)

Task 5: Experiment with Different Learning Rates

Let us train the model with different learning rates and observe their effect on convergence.

```
In [8]: # Train with different learning rates
learning_rates = [0.001, 0.01, 0.1, 0.5]
models = {}

for lr in learning_rates:
    model = LinearRegressionGD(learning_rate=lr, n_iterations=1000)
    model.fit(X_train_processed, y_train.values)
    models[lr] = model
    print(f"Learning Rate {lr}: Final Loss = {model.loss_history[-1]:.4f}")
```

Learning Rate 0.001: Final Loss = 9.1965

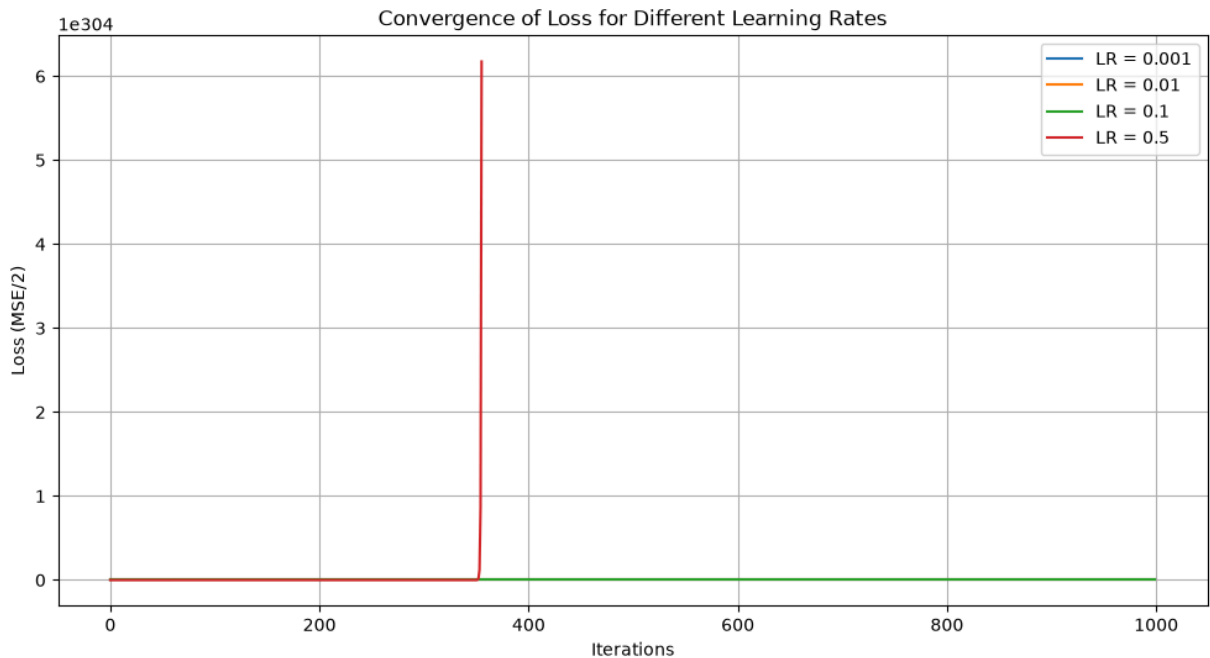
Learning Rate 0.01: Final Loss = 7.8879

Learning Rate 0.1: Final Loss = 7.5020

Learning Rate 0.5: Final Loss = nan

```
In [9]: # Plot loss convergence for different learning rates
plt.figure(figsize=(12, 6))
for lr, model in models.items():
    plt.plot(model.loss_history, label=f"LR = {lr}")

plt.xlabel("Iterations")
plt.ylabel("Loss (MSE/2)")
plt.title("Convergence of Loss for Different Learning Rates")
plt.legend()
plt.grid(True)
plt.show()
```



Task 6: Model Evaluation

We will evaluate the trained model using standard regression metrics:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

```
In [10]: # Select the best model (lowest final loss)
best_lr = min(models.keys(), key=lambda lr: models[lr].loss_history[-1])
best_model = models[best_lr]
print(f"Best learning rate: {best_lr}")

# Make predictions
y_train_pred = best_model.predict(X_train_processed)
y_test_pred = best_model.predict(X_test_processed)

# Calculate metrics
def evaluate_model(y_true, y_pred, dataset_name):
```

```

    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    print(f"\n{dataset_name} Metrics:")
    print(f"    MAE   : {mae:.4f}")
    print(f"    MSE   : {mse:.4f}")
    print(f"    RMSE  : {rmse:.4f}")
    print(f"    R2   : {r2:.4f}")
    return mae, mse, rmse, r2

train_metrics = evaluate_model(y_train.values, y_train_pred, "Training Set")
test_metrics = evaluate_model(y_test.values, y_test_pred, "Testing Set")

```

Best learning rate: 0.1

Training Set Metrics:

```

MAE   : 2.9587
MSE   : 15.0040
RMSE  : 3.8735
R2   : 0.2857

```

Testing Set Metrics:

```

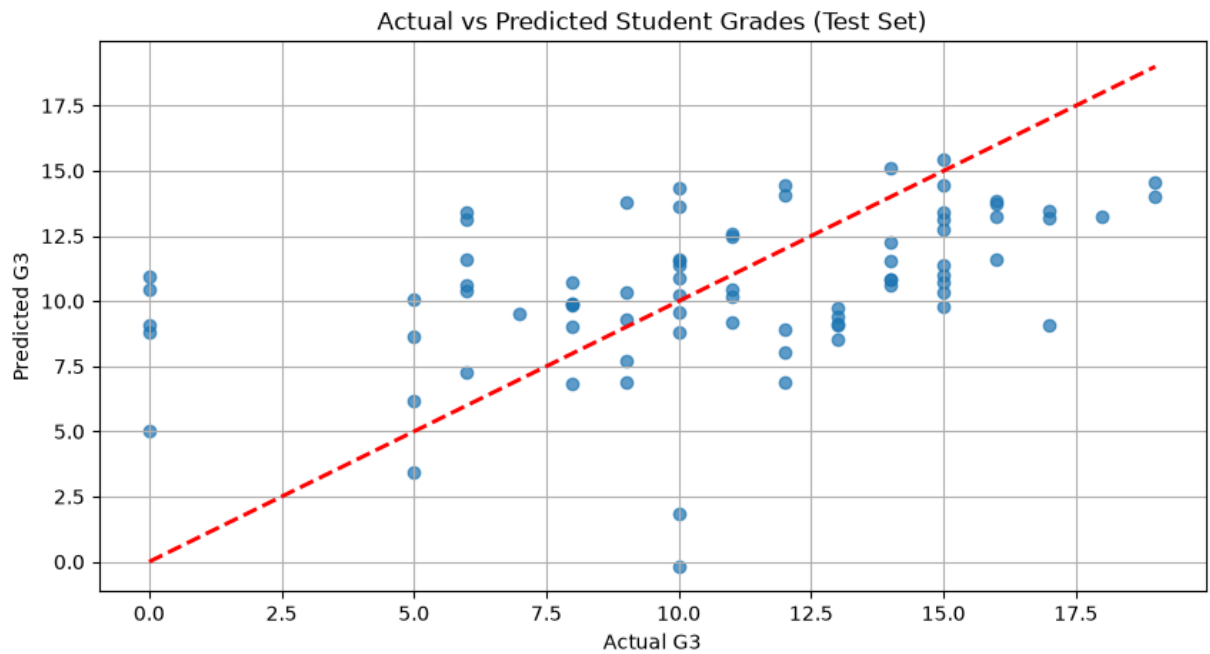
MAE   : 3.4060
MSE   : 17.6304
RMSE  : 4.1989
R2   : 0.1402

```

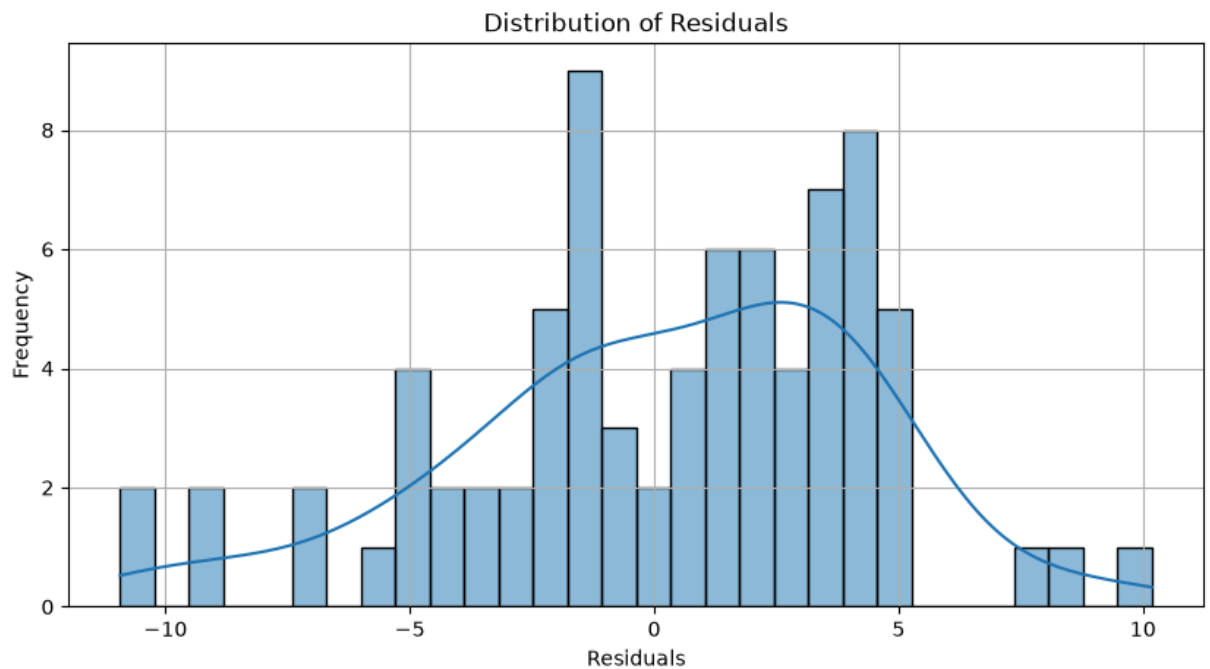
```

In [11]: # Plot actual vs predicted values
plt.figure(figsize=(10, 5))
plt.scatter(y_test, y_test_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--",
plt.xlabel("Actual G3")
plt.ylabel("Predicted G3")
plt.title("Actual vs Predicted Student Grades (Test Set)")
plt.grid(True)
plt.show()

```



```
In [12]: # Plot residual distribution
residuals = y_test.values - y_test_pred
plt.figure(figsize=(10, 5))
sns.histplot(residuals, kde=True, bins=30)
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.title("Distribution of Residuals")
plt.grid(True)
plt.show()
```



Task 7: Interpretation of Results

Convergence Behavior

- A **small learning rate** (0.001) leads to slow convergence and the model may not reach the minimum within the given iterations.
- A **moderate learning rate** (0.01) usually provides stable convergence.
- A **large learning rate** (0.1 or 0.5) may cause the loss to oscillate or even diverge if the rate is too high.

Model Performance

- **R² Score** indicates how well the model explains the variance in the target variable. A value closer to 1 indicates better performance.
- **RMSE** gives an idea of the average error magnitude in the same units as the target variable.
- Comparing training and testing metrics helps identify overfitting or underfitting.

Observations

- The Student Performance dataset is well-structured with no missing values.
- Feature scaling is important for Gradient Descent to converge efficiently.
- The model should achieve reasonable performance on this dataset, as student grades often have strong correlations with study habits and other features.

Task 8: Comparison with Scikit-Learn Implementation

Let us compare our Gradient Descent implementation with Scikit-Learn's Linear Regression for reference.

```
In [13]: from sklearn.linear_model import LinearRegression as SKLinearRegression

# Train scikit-learn model
sk_model = SKLinearRegression()
sk_model.fit(X_train_processed, y_train)

# Predictions
y_test_pred_sk = sk_model.predict(X_test_processed)

# Evaluate
mae_sk = mean_absolute_error(y_test, y_test_pred_sk)
mse_sk = mean_squared_error(y_test, y_test_pred_sk)
rmse_sk = np.sqrt(mse_sk)
r2_sk = r2_score(y_test, y_test_pred_sk)

print("Scikit-Learn Linear Regression Results:")
print(f" MAE : {mae_sk:.4f}")
print(f" MSE : {mse_sk:.4f}")
print(f" RMSE : {rmse_sk:.4f}")
print(f" R2 : {r2_sk:.4f}")
```

```
print(f"\nOur GD Model (LR={best_lr}):")
print(f"  MAE  : {test_metrics[0]:.4f}")
print(f"  MSE  : {test_metrics[1]:.4f}")
print(f"  RMSE : {test_metrics[2]:.4f}")
print(f"  R2  : {test_metrics[3]:.4f}")
```

Scikit-Learn Linear Regression Results:

```
MAE  : 3.3953
MSE  : 17.6037
RMSE : 4.1957
R2  : 0.1415
```

Our GD Model (LR=0.1):

```
MAE  : 3.4060
MSE  : 17.6304
RMSE : 4.1989
R2  : 0.1402
```

Summary

In this lab experiment, we successfully:

1. Downloaded and loaded the Student Performance dataset from the UCI repository.
2. Performed data preprocessing including handling categorical variables and feature scaling.
3. Split the dataset into training (80%) and testing (20%) sets.
4. Implemented Linear Regression using the Gradient Descent algorithm from scratch.
5. Experimented with different learning rates and observed their effect on model convergence.
6. Plotted the loss function versus the number of iterations.
7. Evaluated the model using MAE, MSE, RMSE, and R² Score.
8. Compared our implementation with Scikit-Learn's Linear Regression.

This experiment demonstrates the importance of learning rate selection and feature preprocessing when using Gradient Descent for Linear Regression.